

Project 1 基礎知識ノート

myoArm Forward Model + Kalman-like State Estimation を理解するために

作成日: 2026-05-09 / 対象: 理系大学院生

この資料の位置づけ

Project 1 は、MyoSuite myoArm reaching において「forward model」と「Kalman-like state estimator」が、遅延・観測ノイズ・signal-dependent motor noise 下の制御を改善するかを調べる研究である。本資料は、研究計画書を読み、実装と実験設計に入る前に必要な基礎知識をまとめる。

目次

1. まず何を理解すべきか	1
2. MyoSuite myoArm reaching の基礎	2
2.1. 運動学と動力学	3
3. 信号名の区別	3
4. 状態 x_t と入力 u_t	3
5. Forward model の考え方	4
5.1. one-step と rollout	4
6. 遅延フィードバックと状態推定	5
7. Kalman-like estimator の基礎	5
8. Noise の基礎	5
9. 実験条件と比較表	6
10. Metrics の読み方	6
11. 実装ロードマップ	6
11.1. ActionAdapter から始める理由	7
12. 勉強の順序	7
13. 参考文献・読むべき資料	7
13.1. 図版について	7
14. まとめ	8

1. まず何を理解すべきか

Project 1 の核心は、次の 1 文に集約できる。

Project 1 の中心問い

筋骨格腕 myoArm の reaching で、遅延した感覚フィードバックだけに頼るよりも、内部 forward model による現在状態の予測と、遅延・ノイズ付き観測の統合を使う方が、到達誤差・振動・オーバーシュートを小さくできるか。

この問いを理解するには、最低限次の 5 つを押さえる必要がある。

領域	理解する内容	Project 1 での役割
筋骨格シミュレーション	qpos, qvel, muscle activation, actuator input	plant と真の状態を定義する
制御信号	neural_command, excitation, api_action, ctrl, activation の区別	logger と dataset の混同を防ぐ
forward model	$x_t, u_t \rightarrow x_{t+1}$ または Δx_t	遅延中の現在状態を予測する
状態推定	prediction と observation の統合	Kalman-like estimator の基礎
実験設計	target split, delay/noise, metrics	仮説を反証可能にする

2. MyoSuite myoArm reaching の基礎

MyoSuite は MuJoCo 上で動く筋骨格モデルとタスクの collection であり、Gym / Gymnasium 互換 API で制御実験を行える。Project 1 で主に使う環境は次の 2 つである。

- myoArmReachRandom-v0: target がランダムに変わる reaching task
- myoArmReachFixed-v0: 固定 target への reaching task

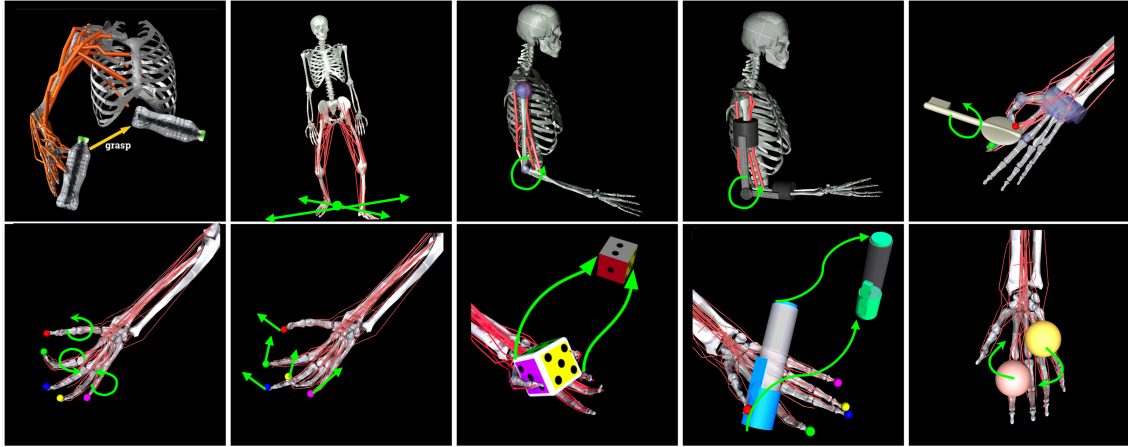


図 1: MyoSuite は複数の筋骨格モデルとタスクを含む suite である。既存 docs/assets 内の公式由来画像を使用。

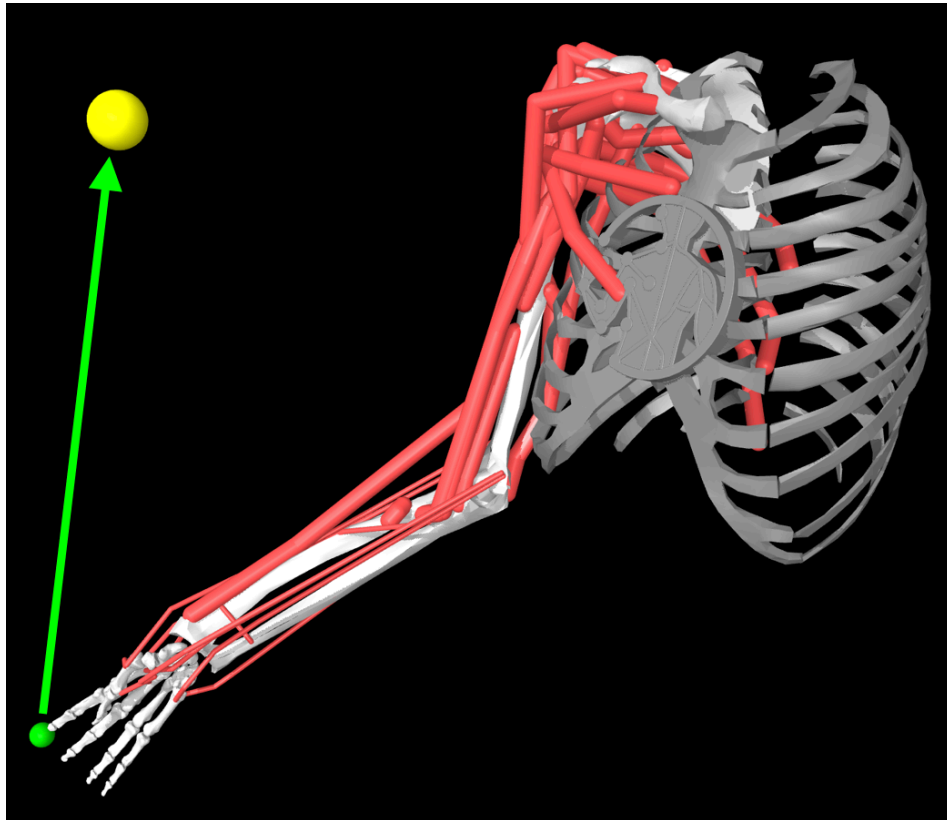


図 2: myoArm reaching task の概念。示指先端を target に近づける。

手元の myosuite 2.12.1 での実測仕様は、既存 primer によれば以下である。

項目	値
観測ベクトル	obs.shape == (80,)
作用空間	Box(-1.0, 1.0, (34,), float32)
MuJoCo nq, nv, nu	20, 20, 34
MuJoCo timestep	0.002 s
MyoSuite frame_skip	10

制御周期 dt	0.02 s
tip site	IFtip
target site	IFtip_target

注意

公式ドキュメントの記述と手元バージョンの actuator 数がずれる可能性がある。研究結果に数値を書くときは、必ず使用した myosuite version と `env.action_space`, `mj_model.nq`, `mj_model.nv`, `mj_model.nu` を実測して記録する。

2.1. 運動学と動力学

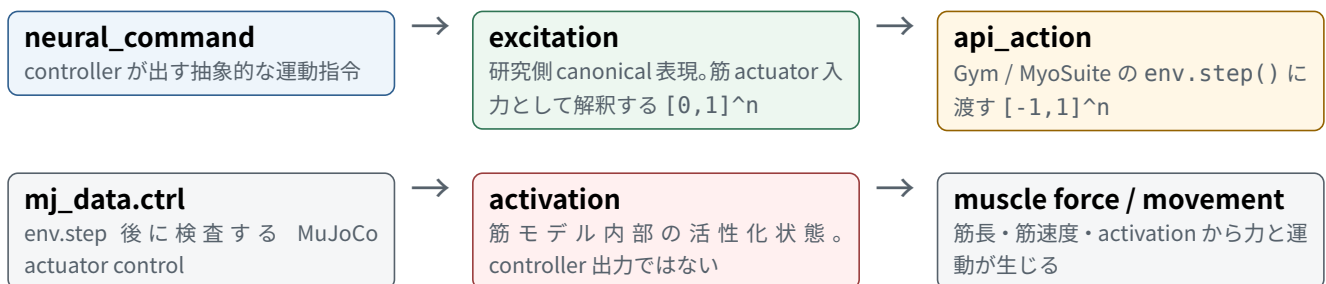
reaching を扱うときは、運動学と動力学を分けると混乱しにくい。

英語	日本語	入力	出力	問うこと
forward kinematics	順運動学	q	手先位置 x	この関節角なら手先はどこか
inverse kinematics	逆運動学	目標手先位置 x^*	関節角 q^*	そこへ手先を置く姿勢は何か
forward dynamics	順動力学	$q, \dot{q}, u$	次状態	この入力で身体はどう動くか
inverse dynamics	逆動力学	$q, \dot{q}, \ddot{q}$	必要な力・トルク	この運動に必要な力は何か

Project 1 の forward model は、順動力学そのものを完全に同定するというより、制御に使える近似モデルとして $x_t, u_t \rightarrow x_{t+1}$ を学習する。

3. 信号名の区別

Project 0 / Project 1 で最も事故りやすいのは、action という言葉を広く使いすぎることである。以下の区別は実装・logger・論文の全てで守る。



ActionAdapter の責務

ActionAdapter は $\text{excitation } [0,1]^n \leftrightarrow \text{api_action } [-1,1]^n$ の薄い変換器である。neural_command $\rightarrow$ excitation は controller 側、activation と mj_data.ctrl は logger / post-step inspection 側の責務とする。

基本変換は次でよい。

```
api_action = 2.0 * clip(excitation, 0.0, 1.0) - 1.0
excitation = (clip(api_action, -1.0, 1.0) + 1.0) / 2.0
```

4. 状態 x_t と入力 u_t

Project 1 の状態表現は、最初は以下を基本にする。

```
x_t = [
    qpos_t,
    qvel_t,
    act_t,
```

```

tip_pos_t,
target_pos_t,
reach_err_t
]

```

ここで重要なのは、controller が見る状態と、評価・学習用に記録する true state を混同しないことである。

量	意味	注意
qpos	関節位置	20 DoF の姿勢
qvel	関節速度	速度は rollout error に効く
act	筋 activation	controller 出力ではなく内部状態
tip_pos	示指先端位置	主要な reaching metric
target_pos	target 位置	target generalization の軸
reach_err	target から tip への誤差	制御則や評価に使う

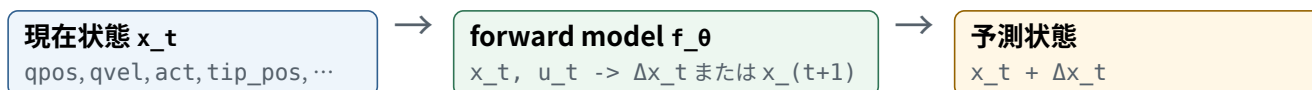
入力 u_t は、学習目的に応じて候補がある。

u_t 候補	利点	注意
excitation	研究側 canonical。生理学的解釈がしやすい	SDN/clipping 後を primary にする
api_action	Gym に渡した値と一致	MyoSuite API 表現であり研究側 canonical ではない
mj_data.ctrl	plant に近い可能性がある	env 内部値であり command と同一視しない

初期方針では、forward model の primary input は SDN / clipping 後の excitation とし、api_action と mj_data.ctrl も logger に保存して後で ablation 可能にする。

5. Forward model の考え方

Forward model は「現在の状態と入力から、次の状態を予測するモデル」である。



最初は残差形式を推奨する。

入力: $[x_t, u_t]$

出力: $\Delta x_t = x_t(t+1) - x_t$

予測: $\hat{x}_t(t+1) = x_t + \Delta \hat{x}_t$

残差形式がよい理由は、制御周期 $dt=0.02$ s では隣接状態が近く、モデルは大きな絶対値ではなく小さな変化量を学習すればよいからである。

5.1. one-step と rollout

Forward model の評価は one-step だけでは不十分である。

評価	意味	注意
one-step prediction	真の x_t から 1 step 先を予測	誤差が小さく見えやすい
10-step rollout	予測値を次の入力として短期展開	制御補償に近い
50-step rollout	長めの自由展開	誤差蓄積と安定性を見る

Project 1 では、MLP baseline から始め、dataset schema と評価が固まった後で GRU / LSTM / Cfc / LTC を比較する。

6. 遅延フィードバックと状態推定

生体制御でもロボット制御でも、感覚フィードバックには遅延がある。myoArm の制御周期は $dt=0.02$ s なので、20 ms の遅延は 1 step、80 ms の遅延は 4 step に相当する。

遅延	steps
0 ms	0
20 ms	1
40 ms	2
80 ms	4
120 ms	6

遅延 feedback の問題は、controller が「過去の身体」を見て現在の身体を制御してしまう点である。



Forward prediction は、遅延中に何が起きたかを内部モデルで補うために使う。

7. Kalman-like estimator の基礎

Kalman filter の本質は、予測と観測を重み付きで統合することである。Project 1 では、厳密な線形ガウス Kalman filter を再現することが目的ではない。筋骨格 plant に対して、同じ発想を使った最小の統合器を作る。

最小式は以下である。

```

x_pred = f(x_est_t, u_t)
y_obs  = delayed_noisy_observation
x_est  = x_pred + K (y_obs - h(x_pred))
  
```

ここで $\text{innovation} = y_{\text{obs}} - h(x_{\text{pred}})$ は、「予測と観測のズレ」である。K が大きければ観測寄り、小さければ予測寄りになる。

K	解釈
0.0	prediction only。観測を使わない
0.25	予測をかなり信じる
0.5	予測と観測を同程度に統合
0.75	観測をかなり信じる
1.0	observation only。予測を補正に使わない

なぜ Kalman-like と呼ぶのか

本物の Kalman filter は状態遷移・観測モデル・共分散更新を明示する。一方、この Project の初期実装では fixed scalar/diagonal gain から始める。数学的には簡略化しているが、prediction と delayed/noisy observation を innovation で統合する点が Kalman 的である。

8. Noise の基礎

Project 1 で扱う noise は大きく 2 種類に分かれる。

種類	入る場所	意味
observation noise	観測 y_t	見えている状態が揺らぐ
signal-dependent motor noise	neural_command / excitation	大きな command ほど motor noise も大きい

Signal-dependent motor noise は、次の形で実装する。

```

u = controller(state)
noise = sigma * abs(u) * rng.normal(size=u.shape)
u_noisy = clip(u + noise, 0.0, 1.0)
api_action = action_adapter.excitation_to_api_action(u_noisy)

```

これは、Gym api_action に直接ノイズを足すよりも、研究側 canonical 表現である excitation にノイズを入れる方が解釈しやすい。

9. 実験条件と比較表

Project 1 の重要な比較条件は以下である。

条件	forward model	delayed obs	gain	目的
Oracle	なし	なし	なし	true state controller の上限
Delayed only	なし	あり	1.0	遅延 feedback の限界
Prediction only	あり	なし	0.0	forward prediction 単独の限界
Fixed Kalman-like	あり	あり	固定	最小統合モデル
Adaptive Kalman-like	あり	あり	noise/delay 依存	不確実性対応
Learned estimator	あり	あり	learned	上限性能

最初から全部を実装しない。Project 0 の共通基盤を作り、Phase 1 の dataset と MLP forward model baseline を固めてから、Phase 2 / 3 に進む。

10. Metrics の読み方

制御性能と推定性能を分けて評価する。

metric	意味	使い方
final tip error	episode 最後の tip-target 距離	最終到達精度
minimum tip error	episode 中の最小距離	一度でも近づけたか
success rate	閾値内到達率	条件間比較の主指標
overshoot	target を通り過ぎる量	delay で悪化しやすい
oscillation index	振動的な補正の強さ	遅延 feedback の不安定性
effort / activation norm	筋活動量	無理な制御を検出
one-step MSE	1 step 予測誤差	model の局所精度
rollout MSE	複数 step 展開誤差	制御に使えるかの目安
state estimation error	推定状態と true state の差	estimator の本質指標

metric 設計の原則

到達性能が改善しても、推定性能が悪ければ「forward model / estimator が効いた」とは言いにくい。逆に推定性能が改善しても、controller がその推定を使わなければ reaching は改善しない。Project 1 では両方を同時に見る。

11. 実装ロードマップ

Project 0 / Phase 0 の最初の到達点は、再現可能なデータセットを作る共通基盤である。

順序	実装	目的
1	ActionAdapter	excitation, api_action, activation, ctrl の混同を防ぐ
2	target set generator	train / validation / test / extrapolation を固定
3	state schema	x_t の shape と dtype を明示

4	SDN / wrappers	motor noise と observation delay/noise を切り替える
5	episode logger	全信号を同じ schema で保存
6	baseline controllers	random / hold / PD で軌跡を集める
7	metrics	制御・推定性能を比較可能にする
8	MLP forward model	Phase 1 baseline

11.1. ActionAdapter から始める理由

ActionAdapter は小さいが、後続の logger、SDN、dataset、forward model の意味を決める。ここで `api_action` と `excitation` を混同すると、後で「何を学習しているのか」「何を制御しているのか」が不明確になる。

実装上の最重要ルール

`activation` は controller 出力ではない。`mj_data.ctrl` は `env.step()` 後に検査する内部 `actuator control` である。ActionAdapter が扱うのは `excitation` と `api_action` の変換だけである。

12. 勉強の順序

理系大学院生がこの Project を読み始めるなら、以下の順序が効率的である。

順序	勉強すること	到達目標
1	MyoSuite / MuJoCo / Gym API	<code>reset</code> , <code>step</code> , <code>action_space</code> , <code>obs_dict</code> が読める
2	筋骨格モデルの基本	<code>excitation</code> , <code>activation</code> , <code>muscle force</code> の違いが分かる
3	状態空間モデル	<code>x_t</code> , <code>u_t</code> , <code>y_t</code> , <code>f</code> , <code>h</code> の意味が分かる
4	forward dynamics / forward model	one-step と rollout 評価を区別できる
5	Kalman filter の直感	<code>prediction</code> , <code>observation</code> , <code>innovation</code> , <code>gain</code> が説明できる
6	実験計画	<code>delay/noise/target split/metrics</code> の必要性が分かる

13. 参考文献・読むべき資料

まずはプロジェクト内資料を読む。

- docs/00_全体研究計画.md
- docs/01_Project1_ForwardStateEstimation研究計画.md
- docs/02_InitialImplementationPlan.md
- docs/myoArm_Reaching_Primer.md
- docs/myoArm_MyoSuite_horizonとmax_steps混同メモ_2026-05-09.md

外部文献としては、以下のキーワードから入るとよい。

- Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems.
- Wolpert, D. M., Ghahramani, Z., & Jordan, M. I. Internal models for sensorimotor integration.
- Harris, C. M., & Wolpert, D. M. Signal-dependent noise determines motor planning.
- Todorov, E., & Jordan, M. I. Optimal feedback control as a theory of motor coordination.
- MuJoCo documentation: actuator, muscle, timestep, control.
- MyoSuite documentation and MyoHub/myosuite repository.

13.1. 図版について

本資料では、外部サイトから新規に画像を取得していない。MyoSuite 関連画像は、既存の `docs/assets/myoarm-reaching-primer/` に保存されていた画像を利用した。既存 primer では MyoHub/myosuite 由来、Apache-2.0 と記録されている。信号フロー、forward model、遅延、Kalman-like estimator については、本文中で自作した模式図を用いた。

14. まとめ

Project 1 は、単なる neural network prediction 実験ではない。筋骨格 plant、遅延した感覚、motor noise、forward prediction、state estimation、closed-loop control を一つの実験系でつなぐ研究である。

最初を守るべきことは、用語と信号の境界である。

最後に

neural_command, excitation, api_action, mj_data.ctrl, activation, true_state, state_estimate を分けて記録・評価できれば、Project 1 の後続実験はかなり健全に進められる。